CS3210 Parallel Computing

AY 25/26 Sem 2 — github/omgeta

1. Introduction

Parallel Computing is the simultaneous use of multiple processing units to solve problems efficiently.

Program Parallelization Steps:

i. Decomposition: split problem into parallel tasks
   - Granularity: size of task

ii. Scheduling: assign tasks to processes/threads
   - Orchestration: imposed synchronisation and communication of tasks to satisfy dependencies

iii. Mapping: bind processes/threads to hardware processing units (e.g. CPU cores)

Processes

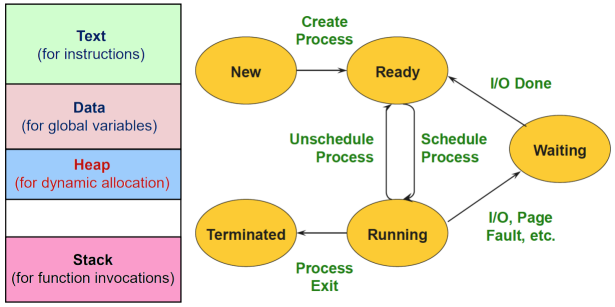
Process is an abstraction for a running program:

i. Process ID (PID): uniquely identifies a process

ii. Process State: indicates execution status

iii. Process Control Block: stores execution context (registers, resources, private address space)

– Costly: syscall overhead, data structures must be allocated, communication goes through OS



Interprocess Communication (IPC):

i. Shared Memory (e.g. locks, semaphores)

ii. Message Passing

iii. UNIX Pipes and Signals

Threads

Threads are independent execution flows within a process:

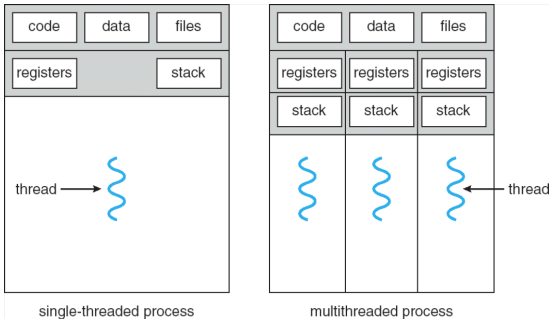
i. Shared: Resources, Address Space

ii. Private: Thread ID, Registers, "Stack" (diff. SP)

\+ Efficient: cheaper creation and context switching

\+ Resource Sharing: process resources can be shared

\+ Concurrent: multithreading on different cores



Implementations:

i. User-Threads are managed by a userspace library:
   + Fast: context switches have low overhead
   – No Parallelism: OS cannot map threads to different execution units
   – Threads blocking will block whole process

ii. Kernel-Threads are implemented in the OS:
   + Parallel: threads can run across many CPUs
   – Slower: thread operations are system calls

Mapping Models:

i. Many-to-One:
   - All user-level threads are mapped to one process
   - User library schedules user-threads

ii. One-to-One:
   - Each user-thread mapped to one kernel-thread
   - No user library scheduler needed

iii. Many-to-Many:
   - User-threads mapped to set of kernel-threads
   - Library scheduler may move user-threads to different kernel-threads

Synchronisation

Synchronisation restricts interleavings of concurrent threads to coordinate access to shared resources.

Critical Section protects resources from concurrent access:

i. Safety: nothing bad happens
   - Mutual Exclusion: at most 1 thread in CS

ii. Liveness: something good happens
   - Progress: if no thread is in CS, a waiting thread should be granted access
   - Bounded Wait: a waiting thread requesting to enter the CS, will eventually enter

iii. Performance: overhead is small w.r.t work done

Symptoms of Incorrect Synchronization:

i. Deadlock: all threads waiting; iff all conditions met
   - Mutual Exclusion: ≥ 1 resource held exclusively
   - Hold & Wait: ≥ 1 process holding one resource while waiting for another
   - No Pre-emption: resources must be yielded
   - Circular Wait: set of processes waiting in circles

ii. Starvation: a thread is unable to make progress
   - Livelock: due to deadlock avoidance

iii. Race Condition: bug from program outcome depending on unpredictable access order

iv. Data Race: two threads access a shared resource with no synchronisation, and one is a write

Mechanisms:

i. Lock: primitive `acquire()`, `release()`
   - Spinlock (busy-wait) vs. Mutex (blocking)
   - Uses hardware atomics (`TSL`) or toggle interrupts

ii. Semaphore: atomic counter ≥ 0; `wait()`, `signal()`
   - Binary/Mutex: represents single access
   - Counting/General: number of threads in CS $\leq N$
   - Can be manipulated by different thread
   - No relation to data being controlled

iii. Monitor: high-level implicit sync data structure

iv. Messages

Producer-Consumer

Processes share a buffer, where producers add items and consumers remove items when not empty.

Unbounded Buffer (`mutex = Sem(1)`, `items = Sem(0)`)

```
// Producer
x = produce();
mutex.wait();
  buffer.add(x);
mutex.signal();
items.signal();
```

```
// Consumer
items.wait();
mutex.wait();
  x = buffer.get();
mutex.signal();
consume(x);
```

Finite Buffer (`spaces = Sem(K)`):

```
// Producer
x = produce();
spaces.wait();
mutex.wait();
  buffer.add(x);
mutex.signal();
items.signal();
```

```
// Consumer
items.wait();
mutex.wait();
  x = buffer.get();
mutex.signal();
spaces.signal();
consume(x);
```

Lightswitch

```
class Lightswitch:
  def __init__(self):
    self.counter = 0
    self.mutex = Sem(1)

  def lock(self, sem):
    self.mutex.wait()
      self.counter++
      if self.counter == 1:
        sem.wait()
    self.mutex.signal()

  def unlock(self, sem):
    self.mutex.wait()
      self.counter--
      if self.counter == 0:
        sem.signal()
    self.mutex.signal()
```

Barrier

```
bool last = false;
mutex.wait();
count++;
if (count == N) {
  last = true;
  barrier.signal();
}
mutex.signal();

barrier.wait();
if (!last)
  barrier.signal();
```

Reader-Writer

Processes share a critical region, where readers can read simultaneously but writers must have exclusive access.

```
// Writer
empty.wait();
  write();
empty.signal();
```

```
// Reader
mutex.wait();
  readers++;
  if (readers == 1)
    empty.wait();
mutex.signal()
read();
mutex.wait();
  readers--;
  if (readers == 0)
    empty.signal();
mutex.signal()
```

Lightswitch:

```
// Writer
empty.wait();
  write();
empty.signal();
```

```
// Reader
rswitch.lock(empty);
  read();
rswitch.unlock(empty);
```

No Starvation (`gate = Sem(1)`):

```
// Writer
gate.wait();
empty.wait();
  write();
gate.signal();
empty.signal();
```

```
// Reader
gate.wait();
gate.signal();
rswitch.lock(empty);
  read();
rswitch.unlock(empty);
```

Priorities:

```
// Writer
wswitch.lock(noRead);
noWrite.wait();
  write();
noWrite.signal();
wswitch.unlock(noRead);
```

```
// Reader
noRead.wait();
rswitch.lock(noWrite);
noRead.signal();
  read();
rswitch.unlock(noWrite);
```

Dining Philosophers

N philosophers around a circular table, with a single chopstick between. 2 chopsticks are needed to eat.

```
// Philosopher
void p(int i) {
  while (1) {
    // think
    take(i);
    eat();
    put(i);
  }
}

void safe_to_eat
    (int i) {
  if (state[i] ==
   HUNGRY &&
   state[LEFT] !=
   EATING &&
   state[RIGHT] !=
   EATING) {
    state[i] =
   EATING;
    signal(s[i]);
  }
}
```

```
// Take Chopstick
void take(int i) {
  wait(mutex);
  state[i] =
   HUNGRY;
  safe_to_eat(i);
  signal(mutex);
  wait(s[i]);
}

// Put Chopstick
void put(int i) {
  wait(mutex);
  state[i] =
   THINKING;

   safe_to_eat(LEFT);

   safe_to_eat(RIGHT)
  signal(mutex);
}
```

Limited Eater Solution, when at most $N-1$ philosophers eat concurrently, it's guaranteed at least one can eat:

```
void philosopher(int i) {
  while (1) {
    // think
    wait(seats);
    wait(chopstick[LEFT]);
    wait(chopstick[RIGHT]);
    eat();
    signal(chopstick[RIGHT]);
    signal(chopstick[LEFT]);
    signal(seats);
  }
}
```

2. Architecture

Levels of Parallelism:

i. Bit-Level: operate on wider words
   - SIMD: operate on superword/vector registers

ii. Instruction-Level:
   - Pipeline: split execution into stages; multiple instructions can occupy different stages in same cycle (if no dependency hazards)
   - Superscalar: duplicate pipelines; issue multiple independent instructions per cycle

iii. Thread-Level: execute multiple threads on the same core in parallel (i.e. SMT)

iv. Processor-Level: execute multiple threads/processes in parallel on multiple cores/processors

Flynn Taxonomy of Parallel Architectures:

i. Single Instruction Single Data (SISD): single stream, each instruction on single data

ii. Single Instruction Multiple Data (SIMD): single stream, each instruction on multiple data

iii. Multiple Instruction Single Data (MISD): multiple streams, each instruction on same data

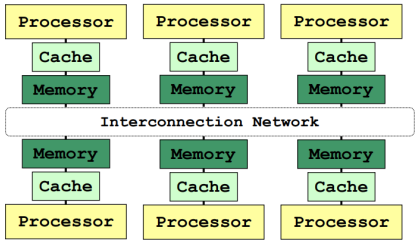
iv. Multiple Instruction Multiple Data (MIMD): per-PU stream to work on its own data

Distributed-Memory

Distributed-Memory Systems are multicomputer systems:

i. Node: indep. unit of processor and private memory

ii. Interconnection Network: supports message passing for data exchange between nodes



Shared-Memory

Shared-Memory Systems are multiprocessor systems:

i. Shared Memory Provider: maintains shared address space abstraction across PUs

\+ No need to partition code or data

\+ Efficient communication: no need to physically move data between processors

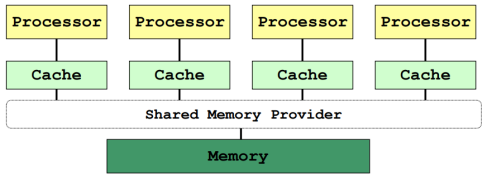
– Special synchonisation constructs required

– Limited scalability due to contention

Uniform Memory Access (UMA):

i. Same memory access latency for every PU

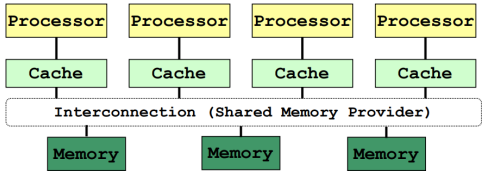
\+ Suitable with few processors due to contention



Non-Uniform Memory Access (NUMA):

i. Different memory access latency for every PU (e.g. local memory faster than remote)

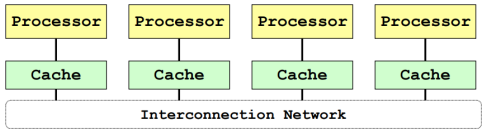
ii. Cache Coherent NUMA (ccNUMA): each node has coherent cache to reduce contention



Cache Only Memory Access (COMA):

i. Memory blocks behave as caches

ii. Data migrates dynamically and continuously according to cache coherence scheme



Cache Coherence

Caches reduce memory access latency and provide high bandwidth data transfer to CPU:

i. Cache Size: larger cache reduces misses but increases access time (addressing complexity)

ii. Block Size: unit of transfer; larger blocks improve spatial locality, but longer transfer time

Write Policy:

i. Write-through: write to both cache and memory
   \+ Always get newest value of memory block
   – Slower due to many memory accesses; mitigate with write buffer

ii. Write-back: write to cache only and set dirty bit; write to memory on cache block eviction
   \+ Fewer write operations to memory
   – Memory may contain invalid entries

Coherence defines behaviour of reads/writes to the same memory location. Properties:

i. Program Order: if P writes to X, n.f.w to X, P should read same value from X

ii. Write Propagation: if P_1 writes to X, n.f.w to X, P_2 should read same value from X eventually

iii. Transaction Serialization: if $V_1 \to X$, then $V_2 \to X$, all P_i should never read V_2 then V_1 from X

Implementations:

i. Software-based: use page-fault to propagate writes

ii. Snooping-based: cache controllers snoop/broadcast on shared bus

iii. Directory-based: central directory records sharing status; no broadcast; common in NUMA

Implications:

i. "Increase" memory latency and lower cache hit rate

ii. Cache Ping-Pong: 2 PUs write to same cache line; invalidations slow fetch from memory

iii. False Sharing: PUs write to different addresses on same cache line; avoid with padding

Memory Consistency

Memory Consistency constrains the order in which memory operations by a thread become visible (commit) to other threads for different memory locations.

Memory Ordering Requirements:

i. $W \to R$: write to X commits before read from Y
ii. $R \to R$: read from X commits before read from Y
iii. $R \to W$: read from X commits before write to Y
iv. $W \to W$: write to X commits before write to Y

Sequential Consistency Model (SC):

i. Preserves all four ordering requirements
ii. PUs issue memory operations in program order
iii. Interleaving of PUs' program orders is acceptable
– Can result in loss of performance

Relaxed Consistency Model:

i. Relax ordering only if no data dependencies between operations to same memory location:
   - $R \to W$: anti-dependence (WAR)
   - $W \to W$: output dependence (WAW)
   - $W \to R$: flow dependence (RAW)
ii. Override constructs can prevent certain re-orderings
iii. Total Store Ordering (TSO):
   - Relax $W \to R$: a PU may read B before its earlier write to A is visible to *all* other PUs
   - Write Atomicity: writes become visible to all PUs at the same time
iv. Processor Consistency (PC):
   - Relax $W \to R$
   - No Write Atomicity: a write may be seen by some PUs before others
v. Partial Store Ordering (PSO):
   - Relax $W \to R$
   - Write Atomicity
   - Relax $W \to W$: a later write to B can bypass an earlier write to A

\+ Hide latencies by overlapping indep. operations

Interconnection Networks

Interconnection forms the backbone of communication between processors, memories/caches, and I/O devices.

Topology

Topology is the graph structure of the network.

Direct Interconnect: endpoints connected directly, usually of same type (e.g. core–core)

i. Diameter, $\delta(G)$: maximum shortest-path distance between any pair of nodes
   - Small diameter $\Rightarrow$ shorter message transmission
ii. Degree, $g(v)$: no. of nodes adjacent to node v; $g(G)$ is the maximum node degree in network G
   - Small degree $\Rightarrow$ lower node hardware overhead.
iii. Bisection Width, $B(G)$: minimum no. of edges removed to split the network into two equal halves
   - Bisection bandwidth $BW(G)$: total bandwidth between the two bisections
   - Measures capacity in the worst case
iv. Node Connectivity, $nc(G)$: minimum no. of nodes whose failure disconnects the network
   - Determines robustness of the network
v. Edge Connectivity, $ec(G)$: minimum no. of edges whose failure disconnects the network
   - Determines number of independent paths between any pair of nodes

	n	$g(G)$	$\delta(G)$	$ec(G)$	$B(G)$
Complete Graph	n	$n-1$	1	$n-1$	$(\frac{n}{2})^2$
Linear Array	n	2	$n-1$	1	1
Ring	n	2	$\lfloor \frac{n}{2} \rfloor$	2	2
d-dim Mesh	r^d	$2d$	$d(\sqrt[d]{n}-1)$	d	$n^{\frac{d-1}{d}}$
d-dim Torus	r^d	$2d$	$d\lfloor \frac{\sqrt[d]{n}}{2} \rfloor$	$2d$	$2n^{\frac{d-1}{d}}$
k-dim Hypercube	2^k	$\log n$	$\log n$	$\log n$	$\frac{n}{2}$
k-dim CCC	$k2^k$	3	$2k-1+\lfloor \frac{k}{2} \rfloor$	3	$\frac{n}{2k}$
Complete BT	2^k-1	3	$2\log\frac{n+1}{2}$	1	1
k-ary d-cube	k^d	$2d$	$d\lfloor \frac{k}{2} \rfloor$	$2d$	$2k^{d-1}$

Indirect Interconnection: endpoints communicate through dynamically configured switches:

\+ Reduce hardware costs by sharing switches & links
\+ Sometimes connect different types of hardware (e.g. PUs and memory modules)
i. Cost: number of switches and links
ii. Concurrent connections

Types:

i. Bus: set of wires carry data from sender to receiver
   - Only a pair of devices can communicate at a time
   - Bus arbiter used for coordination
   - Typically used for small number of processors
ii. Crossbar: switch matrix (of $n \times m$ switches) from n inputs to m outputs
   - Switch states: straight or direction change
   - Hardware costly $\to$ small number of processors
iii. Multistage Switching: intermediate switches with connecting wires between neighbouring stages
   - Goal: obtain small distance for any pair of input and output devices
iv. Omega: unique path from each input to each output ($n \times n$)
   - Construction: switch (pos α, stage i) $\to (\beta, i+1), (\beta', i+1)$ where β is cyclic-left-shift of α, β' is cyclic-left-shift + inversion of LSB
   - $\log n$ stages of $\frac{n}{2}$ switches per stage
   - Connections between stages are regular $\to$ good for manufacturing
v. Butterfly:
   - Construction: switch $(\alpha, i) \to (\alpha, i+1), (\alpha', i+1)$ where α, α' differ in the $(i+1)^{th}$ bit from the left
vi. Baseline:
   - Construction: switch $(\alpha, i) \to (\beta, i+1), (\beta', i+1)$ where β is cyclic-right-shift of last $(k-i)$ bits, β' is inversion of LSB + same cyclic-right-shift

Routing

Routing determines path(s) from source to destination within a given topology. Algorithm Classification:

i. Path Length:
   - Minimal: shortest-path always chosen
   - Non-minimal: shortest-path not necessary

ii. Adaptivity:
   - Deterministic: always use same path for same pair of source and destination nodes
   - Adaptive: take into account network status (e.g. avoid congested path, avoid dead nodes)

Deterministic Algorithms:

i. XY Routing for 2D Mesh:
   - Move in X direction until $X_{src} == X_{dst}$
   - Move in Y direction until $Y_{src} == Y_{dst}$

ii. E-Cube Routing for Hypercube:
   - Bit difference in source and target node address (hamming distance) = number of hops
   - From MSB→LSB (or vice versa), find first different bit and go to neighbour node with bit corrected; at most dim-k hops

iii. XOR-Tag Routing for Omega Network:
   - Let T = Source ID $\oplus$ Destination ID
   - At stage k, go straight if bit $k = 0$, else crossover

Data Distribution

Patterns:

i. Blockwise: contiguous chunks per processor; good for spatial locality.

ii. Cyclic: round-robin elements per processor; good for load balance.

iii. Block-cyclic: cyclic distribution of blocks of size b; balances locality and load.

Choose distribution based on which processors need which data.

3. Programming Models

Types of Parallelism:

i. Data Parallelism: split data among PUs (loop parallelism; SIMD/SPMD)

ii. Task Parallelism: split indep. tasks among PUs (statements, loops, function calls)
   - Task Dependency Graph: DAG representing control dependency between tasks
     - Critical Path Length: slowest completion
     - Deg. Concurrency: $\dfrac{\text{Total Work}}{\text{Critical Path Length}}$

Representation of Parallelism:

i. Implicit Parallelism: Haskell

ii. Implicit Scheduling: OpenMP

iii. Explicit Communication & Sync: MPI, Pthreads

Models of Coordination:

i. Shared Address Space: very little structure
   - All threads can access all shared variables; mutual exclusion via locks
   - Match shared-memory systems (UMA/NUMA); requires HW support; costly to scale
   - \+ Simple to program, if synced correctly
   - − Scalability limited by memory cost variation (even with NUMA) or bandwidth limiting speed

ii. Message Passing: highly structured
   - Tasks operate in private address spaces; communication via explicit send/receive
   - Matches distributed memory systems
   - \+ Allows scaling across many machines
   - − Complex programming
   - − Network communication delays

Foster's Design Methodology

i. Partitioning: problem into pieces/data
   - $\geq 10\times$ primitive tasks than cores, of similar size
   - Minimize redundant computations and data
   - Number of tasks increases with problem size

ii. Communication: pass needed data among tasks
   - Balance communication among tasks
   - Perform communication in parallel
   - Overlap computation with communication
   - Communicate with small number of neighbours

iii. Agglomeration: combine tasks into larger tasks
   - Number of tasks $\geq$ number of cores
   - Reduce cost of task creation + communication
   - Maintain scalability, simplify code

iv. Mapping: tasks to execution units
   - Maximize processor utilization; minimize IPC
   - Optimal mapping is NP-hard ⇒ heuristics
   - Consider static vs dynamic allocation; if static, tasks:cores $\geq$ 10:1; if dynamic, allocator must not bottleneck

Parallel Programming Patterns

Patterns:

i. Fork–Join: `fork` child tasks, then `join` (D&C)

ii. Parbegin–Parend: mark statements to execute in parallel threads, then sync at end

iii. SIMD: single instruction stream over multiple data elements (synchronous)

iv. SPMD: same program on multiple threads; different data via IDs (asynchronous)

v. Master–Worker: master distributes tasks to workers

vi. Task Pools: shared pool of tasks; fixed number of worker threads dynamically dequeue/insert tasks

vii. Producer–Consumer: produce tasks into buffer for consumers to retrieve; sync coordinates passing

viii. Pipelining: split compute into stages for data stream; throughput improves via stage-overlap

OpenMP

Shared-Memory Programming Model:

i. Threads communicate via shared variables; may also have private variables
ii. Avoid race conditions using mutual exclusion

Constructs

General:

i. `#pragma omp parallel`: define parallel region (fork–join), creating team of threads in the block
ii. `omp_get_thread_num()`/`omp_get_num_threads()`: query a thread's id and the current team size.
iii. `omp_set_num_threads(n)` / `OMP_NUM_THREADS`: control the number of threads

Data Parallel For:

i. `#pragma omp for`: work-sharing for loops
ii. `schedule(kind,size)`: static vs. dynamic allocation and chunking
   - Default: static, blockwise
iii. `shared(...)`/`private(...)`: data-scoping clauses
   - Default: variables declared outside shared; variables declared inside private
iv. `reduction(op:var)`: private per-thread value, combined at end.
v. `collapse(n)`: flatten `n` nested loops for iterations

Controlled:

i. `#pragma omp sections`/`#pragma omp section`: split independent blocks among threads
ii. `#pragma omp single`: one thread executes block
iii. `#pragma omp master`: only master executes block
   - Default: no implicit barrier

Synchronisation:

i. `#pragma omp barrier`: explicit barrier for threads
ii. `#pragma omp critical`: critical section block
iii. `#pragma omp atomic`: atomic update to a location
iv. `nowait`: disable implicit barrier after blocks

Parallel Region

```
omp_set_num_threads(4);
#pragma omp parallel
{
  int tid = omp_get_thread_num();
  int nt  = omp_get_num_threads();
  printf("Thread %d out of %d\n", tid, nt);
}
```

Parallel For

```
// a, b, c, N are shared by all threads.
// i and tmp are private to each thread.
// dynamic scheduling assigns chunks of 8
    iterations at a time.
#pragma omp parallel for \
  shared(a, b, c, N) \
  private(i, tmp) \
  schedule(dynamic, 8)
for (int i = 0; i < N; i++) {
  int tmp = a[i] + b[i];
  c[i] = tmp;
}
```

Reduction

```
int sum = 0;
#pragma omp parallel for \
  reduction(+:sum)
for (int i = 0; i < N; i++) {
  sum += a[i];
}
```

Collapse

```
#pragma omp parallel for \
  collapse(2)
for (int i = 0; i < num_rows; i++) {
  for (int j = 0; j < num_cols; j++) {
    C[i][j] = A[i][j] + B[i][j];
  }
}
```

Mutex

```
omp_lock_t lock;
omp_init_lock(&lock);
#pragma omp parallel
{
  omp_set_lock(&lock);
  x++;
  omp_unset_lock(&lock);
}
```

Sections

```
#pragma omp parallel sections
{
  #pragma omp section
  {
    read_input();
  }

  #pragma omp section
  {
    initialize_table();
  }
}
```

Single and Master

```
#pragma omp parallel
{
  #pragma omp single
  {
    init_shared_data();
  }

  #pragma omp master
  {
    printf("Only thread 0 runs this block.\n");
  }
}
```

Critical & Atomic

```
int x = 0;
int y = 0;
#pragma omp parallel
{
  #pragma omp atomic
  x++;

  #pragma omp critical
  {
    y++;
    printf("y = %d\n", y);
  }
}
```

NVIDIA CUDA

CUDA Programming Model:

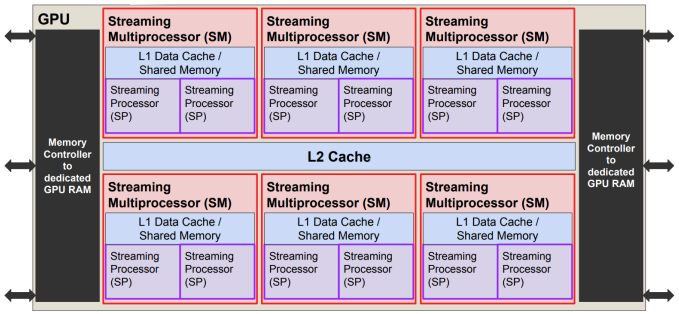
i. Heterogeneous: Host (CPU) + Device (GPU)
ii. Kernel: device function launched by host
iii. Thread: smallest execution unit, with private registers/local variables
iv. Block: group of threads that can cooperate via shared memory and `__syncthreads()`
v. Grid: all blocks launched by a kernel; no synchronization across blocks

Function Qualifiers

Qualifier	Scope	Caller
`__host__`	CPU	CPU
`__device__`	GPU	GPU
`__global__`	GPU	CPU

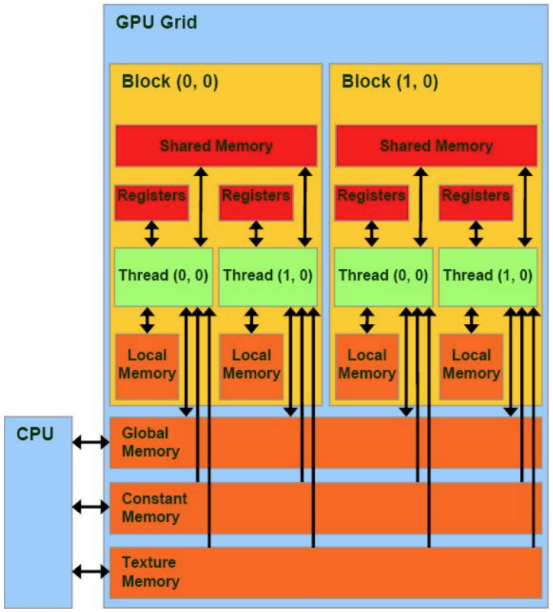
CUDA Execution Model:

i. Streaming Multiprocessor (SM): schedules resident thread blocks which do not migrate
   - Streaming Processor (SP): executes warps
   - Multiple blocks may reside on one SM
ii. Warp: group of 32 threads from same block
   - SIMT: threads in a warp execute same instruction in lockstep when possible
   - Divergence: branches within a warp reduce throughput, due to stalls for SIMT
   - Scheduling: ready warps context switched to hide latency if resources allow; registers for assigned warps are already in the register file



CUDA Memory Model:

i. Registers: thread-private, fast and limited
ii. Local Memory: thread-private, slow, cached
   - Used for large data/ arrays
iii. Shared Memory: block-private, fast
   - Programmer-managed
   - Requires `__syncthreads()` for synchronization
iv. Global Memory: global, read-write, slow, cached
   - Usage: `cudaMalloc`, `cudaMemcpy`, `cudaFree`
v. Constant Memory: global, read-only, slow, cached
   - Better for linear access
vi. Texture Memory: global, read-only, slow, cached
   - Better for 2D spatial access



Variable Qualifiers

Qualifier	Memory	Scope	Lifetime
auto non-array	register	thread	kernel
auto array	local	thread	kernel
`__device__ __constant__`	constant	grid	static
`__device__`	global	grid	static
`__device__ __shared__`	shared	block	kernel
`__managed__`	global	global	static

Coalesced Global Memory Access:

i. Global memory requests are served in fixed-size blocks of 32 bytes (e.g. `float` = 4 bytes)
ii. Coalesced: adjacent warp threads access adjacent addresses → combined into fewer transactions
iii. Memory Utilization: $\frac{\text{Useful Bytes Requested}}{\text{Total Bytes Requested}}$

Shared Memory Bank Conflicts:

i. Shared memory cyclic divided into 32 banks that can be accessed in parallel
ii. No Conflict: warp threads access different banks → parallel access
iii. Bank Conflict: multiple warp threads access different addresses in same bank → serialized access
iv. Broadcast: multiple warp threads access same address → served efficiently

Optimizations:

i. Minimize host–device data transfers
ii. Overlap compute: `cudaMemcpyAsync` + streams
iii. Minimize global memory access w/ shared memory
   - Coalesce global memory accesses
   - Minimize shared memory bank conflicts
iv. Balance Occupancy and Utilization: many warps hide latency; many blocks in parallel; but too many warps and data may not fit into registers
v. Minimize warp divergence
vi. Prefer high-throughput ops: single precision faster, replace div/mod with bitwise ops

Kernel Launch and Indexing

```
__global__
void addVec(float *a, float *b, float *c, int N) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;

  if (i < N)
    c[i] = a[i] + b[i];
}

int threads = 256;
int blocks = (N + threads - 1) / threads;

addVec<<<blocks, threads>>>(a_d, b_d, c_d, N);
cudaDeviceSynchronize();
```

Shared Memory

```
__global__
void copyTile(float *in, float *out) {
  __shared__ float tile[256];

  int i = blockIdx.x * blockDim.x + threadIdx.x;
  int t = threadIdx.x;

  tile[t] = in[i];
  __syncthreads();

  out[i] = tile[t];
}
```

Host–Device Memory Transfer

```
float *a_d, *b_d, *c_d;

cudaMalloc(&a_d, N * sizeof(float));
cudaMalloc(&b_d, N * sizeof(float));
cudaMalloc(&c_d, N * sizeof(float));

cudaMemcpy(a_d, a_h, N * sizeof(float),
    cudaMemcpyHostToDevice);
cudaMemcpy(b_d, b_h, N * sizeof(float),
    cudaMemcpyHostToDevice);

addVec<<<blocks, threads>>>(a_d, b_d, c_d, N);
cudaDeviceSynchronize();

cudaMemcpy(c_h, c_d, N * sizeof(float),
    cudaMemcpyDeviceToHost);

cudaFree(a_d);
cudaFree(b_d);
cudaFree(c_d);
```

2D Grid and Block Indexing

```
__global__
void addMatrix(float *A, float *B, float *C, int
    N) {
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;
  int idx = row * N + col;

  if (row < N && col < N)
    C[idx] = A[idx] + B[idx];
}

dim3 threads(16, 16);
dim3 blocks((N + threads.x - 1) / threads.x,
            (N + threads.y - 1) / threads.y);

addMatrix<<<blocks, threads>>>(A_d, B_d, C_d, N);
```

Shared Memory Padding

```
__global__
void transposePadded(float *A, float *B, int N) {
  __shared__ float tile[16][18];

  int row_r = blockIdx.y * blockDim.y +
    threadIdx.y;
  int col_r = blockIdx.x * blockDim.x +
    threadIdx.x;

  int row_w = blockIdx.x * blockDim.x +
    threadIdx.y;
  int col_w = blockIdx.y * blockDim.y +
    threadIdx.x;

  if (row_r < N && col_r < N)
    tile[threadIdx.y][threadIdx.x] = A[row_r * N
    + col_r];

  __syncthreads();

  if (row_w < N && col_w < N)
    B[row_w * N + col_w] =
    tile[threadIdx.x][threadIdx.y];
}
```

Atomic Add

```
__global__
void countPositive(float *a, int *count, int N) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;

  if (i < N && a[i] > 0)
    atomicAdd(count, 1);
}
```

Reduction Sum

```
__global__
void reduceSumAtomic(float *in, float *out, int
    N) {
  __shared__ float sdata[256];

  int tid = threadIdx.x;
  int i = blockIdx.x * blockDim.x + threadIdx.x;

  sdata[tid] = (i < N) ? in[i] : 0.0f;
  __syncthreads();

  for (int stride = blockDim.x / 2; stride > 0;
    stride /= 2) {
    if (tid < stride)
      sdata[tid] += sdata[tid + stride];

    __syncthreads();
  }

  // cudaMemset(out, 0, sizeof(float)) in host
  if (tid == 0)
    atomicAdd(out, sdata[0]);
}
```

CUDA Streams

```
cudaStream_t s0, s1;
cudaStreamCreate(&s0);
cudaStreamCreate(&s1);

cudaMemcpyAsync(a0_d, a0_h, bytes,
                cudaMemcpyHostToDevice, s0);
kernel<<<blocks, threads, 0, s0>>>(a0_d);

cudaMemcpyAsync(a1_d, a1_h, bytes,
                cudaMemcpyHostToDevice, s1);
kernel<<<blocks, threads, 0, s1>>>(a1_d);

cudaDeviceSynchronize();

cudaStreamDestroy(s0);
cudaStreamDestroy(s1);
```

OpenMPI

Message Passing Model:

i. Distributed Memory: indep. address spaces
ii. Explicit communication with send/recieve
iii. No data races on process-local variables
iv. Communication races/deadlocks still possible

MPI Program Overview:

i. Initialize only once before any other calls
   - `MPI_Init(int *argc, char **argv)`
ii. Communicate to coordinate computation
iii. Exit from message-passing system
   - `MPI_Finalize(void)`
   - `MPI_Abort(MPI_Comm comm, int errorcode)`

Communication Semantics

Buffering: whether send data can be copied to an internal system buffer to avoid idling while waiting for receive

i. Buffered: less idling; buffer overhead
ii. Non-buffered: idling + deadlock risks if mismatch

Blocking: whether resources/buffers are safe to reuse on function call return

i. Blocking: return ⇒ safe to reuse resources/buffers
ii. Non-Blocking: may return before completion
   - `MPI_Test(MPI_Request *r, int *flag, MPI_Status *s)`; `flag` marks completion
   - `MPI_Wait(MPI_Request *r, MPI_Status *s)`

Synchronous: whether sends require a receive to start participating before returning

i. Synchronous: operation does not complete before both sides have started
ii. Asynchronous: sender can proceed without coordination with receiver

Point-to-Point Communication

Messaging Calls:

i. `MPI_Send(void* buf, int count, MPI_Datatype dt, int dst, int tag, MPI_Comm c)`
ii. `MPI_Recv(void* buf, int count, MPI_Datatype dt, int src, int tag, MPI_Comm c, MPI_Status *status)`

Rank Calls:

i. `MPI_Comm_size(MPI_Comm comm, int *size)`: no. of processes in communicator
ii. `MPI_Comm_rank(MPI_Comm comm, int *rank)`: current process rank

Messages Format:

i. Data: start-buffer + count + datatype
ii. Envelope: dst/src (rank) + tag + communicator
iii. Receive buffer must be ≥ message length
iv. Wildcards: `MPI_ANY_SOURCE`, `MPI_ANY_TAG`
v. `MPI_Status`: `MPI_SOURCE`, `MPI_TAG`, `MPI_ERROR`

Ordering:

i. Same sender–receiver: sends are non-overtaking
ii. Different tags: may be received out of send order if receives request specific tags
iii. > 2 processors: undefined order

Deadlocks

Deadlock Types:

i. Message Order: both sides wait on receives
ii. Buffer: no/small buffers ⇒ sends cannot complete

Secure MPI program: correctness does not depend on assumptions about MPI runtime properties.

- Specify send/receive order (e.g. even: send→recv, odd: recv→send)

Collective Communication

Process Groups are ordered sets of processes:

i. Unique rank identifier, starting from 0
ii. Processes may be in multiple groups
iii. Communicator: communication domain for a group
   - Intra-communicator: collectives in a group; default `MPI_COMM_WORLD`
   - Inter-communicator: P2P between two groups
iv. `MPI_Comm_create`, `MPI_Comm_comp`, `MPI_Comm_dupl`, `MPI_Comm_split`, `MPI_Comm_free`

\+ Faster if fewer ranks involved in the subset

Virtual Topologies:

i. Cartesian: `MPI_Cart_create/get/coords/shift`, `MPI_Cartdim_get`
ii. Graph: `MPI_Graph_create/get/neighbours_count/neighbours`, `MPI_Graphdims_get`

Common Collectives:

i. `MPI_Barrier(MPI_Comm comm)`
ii. `MPI_Bcast(void *buf, int count, MPI_Datatype dt, int root, MPI_Comm comm)`
iii. `MPI_Scatter(void *sbuf, int scount, MPI_Datatype stype, void *rbuf, int rcount, MPI_Datatype rtype, int root, MPI_Comm comm)`
iv. `MPI_Gather(void *sbuf, int scount, MPI_Datatype stype, void *rbuf, int rcount, MPI_Datatype rtype, int root, MPI_Comm comm)`
v. `MPI_Reduce(void *sbuf, void *rbuf, int count, MPI_Datatype dt, MPI_Op op, int root, MPI_Comm comm)`
vi. `MPI_Allgather(void *sbuf, int scount, MPI_Datatype stype, void *rbuf, int rcount, MPI_Datatype rtype, MPI_Comm comm)`
vii. `MPI_Reduce_scatter(void *sbuf, void *rbuf, int rcounts[], MPI_Datatype dt, MPI_Op op, MPI_Comm comm)`
viii. `MPI_Alltoall(void *sbuf, int scount, MPI_Datatype stype, void *rbuf, int rcount, MPI_Datatype rtype, MPI_Comm comm)`

Basic MPI Program

```
int rank, size;

MPI_Init(&argc, &argv);

MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
printf("Rank %d of %d\n", rank, size);

MPI_Finalize();
```

Send and Receive

```
int msg;
MPI_Status status;

if (rank == 0) {
  msg = 42;
  MPI_Send(&msg, 1, MPI_INT, 1, 0,
    MPI_COMM_WORLD);
} else if (rank == 1) {
  MPI_Recv(&msg, 1, MPI_INT, 0, 0,
           MPI_COMM_WORLD, &status);
}
```

Wildcard Receive

```
int msg;
MPI_Status status;

MPI_Recv(&msg, 1, MPI_INT,
         MPI_ANY_SOURCE, MPI_ANY_TAG,
         MPI_COMM_WORLD, &status);

printf("from %d tag %d\n",
       status.MPI_SOURCE, status.MPI_TAG);
```

Non-Blocking Send/Receive

```
MPI_Request reqs[2];

MPI_Irecv(&recvbuf, 1, MPI_INT, src, 0,
          MPI_COMM_WORLD, &reqs[0]);

MPI_Isend(&sendbuf, 1, MPI_INT, dst, 0,
          MPI_COMM_WORLD, &reqs[1]);

MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);
```

Deadlock Avoidance

```
MPI_Status status;

if (rank % 2 == 0) {
  MPI_Send(&sendbuf, 1, MPI_INT, right, 0, comm);
  MPI_Recv(&recvbuf, 1, MPI_INT, left, 0,
           comm, &status);
} else {
  MPI_Recv(&recvbuf, 1, MPI_INT, left, 0,
           comm, &status);
  MPI_Send(&sendbuf, 1, MPI_INT, right, 0, comm);
}
```

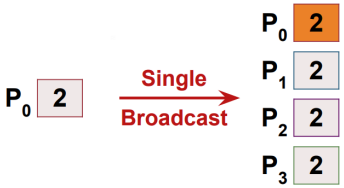
Communicator Split

```
MPI_Comm subcomm;
int color = rank % 2;
int subrank;

MPI_Comm_split(MPI_COMM_WORLD, color, rank,
    &subcomm);
MPI_Comm_rank(subcomm, &subrank);

/* communicate within subcomm */

MPI_Comm_free(&subcomm);
```

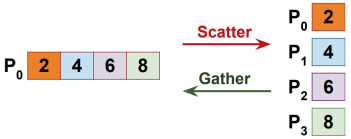
Cartesian Topology

```
MPI_Comm cart;
int dims[2] = {rows, cols};
int periods[2] = {0, 0};

MPI_Cart_create(MPI_COMM_WORLD, 2, dims,
                periods, 0, &cart);

MPI_Cart_shift(cart, 0, 1, &up, &down);
MPI_Cart_shift(cart, 1, 1, &left, &right);

MPI_Comm_free(&cart);
```

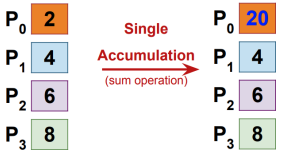
Broadcast



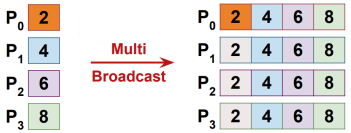
Scatter and Gather



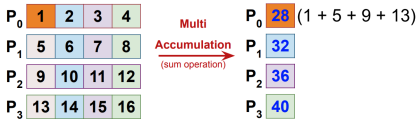
Reduce



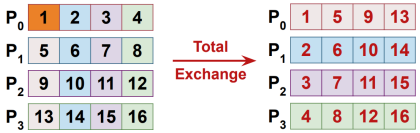
Allgather



Reduce Scatter



All-to-All



4. Performance

Performance is measured by primary goals:

i. Response Time: wall-clock duration of an execution consists of user, system and waiting time
ii. Throughput: average work completed per unit time

Sequential Programs

User CPU Time:

$$T_{user} = N_{cycle} \times T_{cycle}$$

i. $T_{cycle} = \frac{1}{\text{(average) clock freq.}}$
ii. $N_{cycle} = \sum n_i \times CPI_i \approx N_{instr} \times CPI$

User CPU Time (Memory Access; One-level Cache):

$$T_{user} = \left(N_{cycle} + N_{mm_cycle}\right) \times T_{cycle}$$

i. $N_{mm_cycle} = N_{read_cycle} + N_{write_cycle}$
ii. $N_{read_cycle} = N_{reads} \times R_{miss_rate} \times N_{miss_cycles}$

Average Memory Access Time (Two-level Cache):

$$T_{read} = T_{hit} + \left(R_{miss} \times T_{miss}\right)$$

i. $T_{read} = T_{hit}^{L1} + \left(R_{miss}^{L1} \times T_{miss}^{L1}\right)$
ii. $T_{miss}^{L1} = T_{hit}^{L2} + \left(R_{miss}^{L2} \times T_{miss}^{L2}\right)$
iii. Global miss rate: $R_{miss} = R_{miss}^{L1} \times R_{miss}^{L2}$

User CPU Time (Integrated CPI + Memory):

$$T_{user} = \Big(N_{cycle} + [N_{rw_ops} \times R_{miss} \times N_{miss_cycles}]\Big) \times T_{cycle}$$

Throughput Metrics:

i. MIPS $= \frac{N_{instr}}{T_{user} \times 10^6}$
ii. MFLOPS $= \frac{N_{fl_ops}}{T_{user} \times 10^6}$
   + Restrict to float ops (but variance exists)
iii. Arithmetic Intensity (AI) $= \frac{N_{fl_ops}}{N_{bytes_rw}}$
   + Roofline: $P \leq \min\left(P_{\text{peak}},\ B \cdot \text{AI}\right)$ shows if bound by computation or communication bandwidth

Parallel Programs

Parallel Execution Time on p PUs, T_p consists:

i. Time for executing local computations
ii. Time for exchange of data between PUs
iii. Time for synchronization between PUs
iv. Waiting time

Speedup:

$$S_p = \frac{T_*}{T_p}$$

i. Measures the benefit of parallelism compared to best sequential algorithm time T_*
ii. Theoretically, $S_p \leq p$ always holds
iii. In practice, $S_p > p$ (superlinear speedup) can occur (e.g. problem working set "fits" in the cache)

Cost:

$$C_p = p \times T_p$$

i. Measures total work performed by all PUs i.e. processor-runtime product
ii. Cost-optimal: parallel program executes same total number of operations as fastest sequential program

Efficiency:

$$E_p = \frac{T_*}{C_p} = \frac{S_p}{p} = \frac{T_*}{p \times T_p}$$

i. Measures actual degree of speedup performance achieved compared to maximum
ii. Theoretically, ideal $S_p = p \ \Rightarrow\ E_p = 1$
iii. In practice, $E_p > 1$ can occur with superlinear speedups

Scalability

Scalability is the interaction between problem size and parallel machine size:

i. Impacts: load balancing, overhead, arithmetic intensity, locality (application dependent)
ii. Fixed problem size:
   - Small n: overheads dominate $\Rightarrow$ suitable for small machine, poor on large machine
   - Large n: working set may not fit on small machine (thrashing / exceeds cache / can't run)
iii. Constraints:
   - Application: particles/processor (N-body), transactions/processor (distributed DB)
   - Resources: time-limited, memory-limited, problem-limited (same problem faster)

Amdahl's Law:

$$S_p(n) = \frac{T_*(n)}{f \cdot T_*(n) + \frac{1-f}{p} T_*(n)} = \frac{1}{f + \frac{1-f}{p}} \leq \frac{1}{f}$$

i. Sequential fraction: $0 \leq f \leq 1$ limits speedup
ii. Discourage from making large parallel computers, focus on reducing sequential fraction f
iii. Rebuttal: circumvented for large problem size
   - Commonly f is a function of n, $f(n)$
   - Efficient parallel algorithm: $\lim_{n\to\infty} f(n) = 0$
   - Thus, $\lim_{n\to\infty} S_p = \frac{p}{1+(p-1)f(n)} = p$

Gustafson's Law:

$$S_p(n) = \frac{\tau_f + \tau_v(n,1)}{\tau_f + \tau_v(n,p)} = f + (1-f)p$$

i. Encouraged to use more processors to solve larger problem in the same time (in CPU-bound apps)
ii. τ_f: constant sequential execution time
iii. $\tau_v(n,p)$: parallel execution time on p processors; if perfectly parallelizable (no overheads):
   - $\tau_v(n,1) = T^*(n) - \tau_f$
   - $\tau_v(n,p) = \frac{T^*(n) - \tau_f}{p}$
iv. If $T^*(n)$ increases strongly monotonically with n, then $\lim_{n\to\infty} S_p(n) = p$

Instrumentation

Performance Instrumentation finds performance metrics:

i. Latency: time to service a request
ii. Response Time: wall-clock duration of an execution
iii. Throughput: average work completed per unit time
iv. IOPS: I/O operations per second
v. Utilization: fraction of time a resource is busy
vi. Saturation: degree to which a resource has queued work it cannot service

Perspectives:

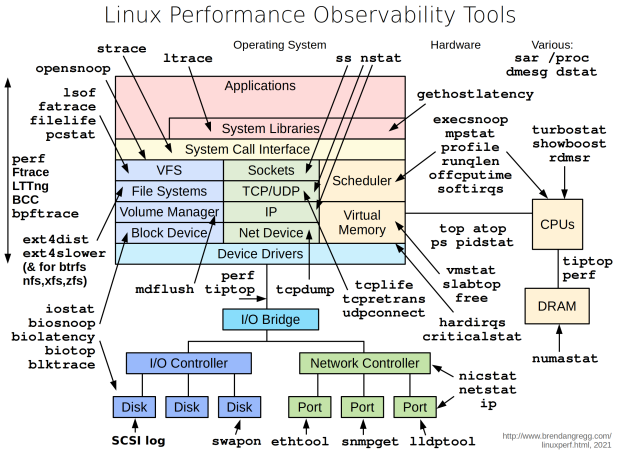
i. Resource Analysis: focus on utilization
   - Time-based: average time the resource was busy
   - Capacity-based: resource throughput
ii. Workload Analysis: focus on throughput, latency

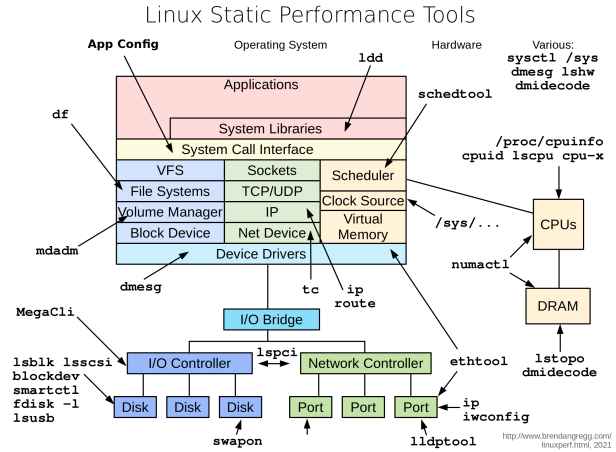
Methodologies:

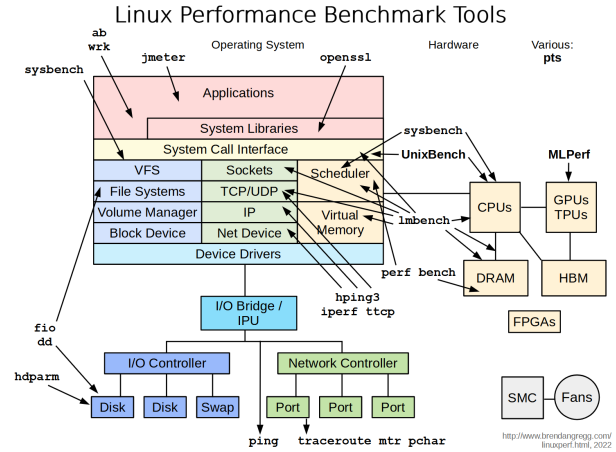
i. Anti-Methodologies: undeliberate, street light (look for obvious issues), drunk man (tune at random)
ii. Problem Statement
iii. USE (per resource): Utilization, Saturation, Errors
iv. Monitoring: record metrics over time

Tool Types & Counters:

i. Observability: watch activity under workload (timing statements, performance counters)
ii. Static: examine system at rest
iii. Benchmarking: load test (caution: contention/production risk)
iv. Tuning: change defaults (danger: regressions now/later under load)
v. Categorization:
   - System-wide vs per-process
   - Fixed counters (kernel-maintained metrics) vs event-based counters (enabled as needed)
   - Profiling: samples/snapshots vs Tracing: record every event occurrence







5. Energy-Efficient Computing

Motivation:

i. Modern compute uses too much power
ii. Higher performance ⇒ more/faster computers ⇒ more power, heat, cooling, space, cost

Energy and Power:

i. Energy E [Joules (J)]: capacity for doing work
ii. Power P [Watts = J/s]: energy transferred/converted per unit time

$$P = \frac{E}{t}$$

iii. Higher energy consumption costs more
iv. Almost ∼100% processor power becomes heat

Power Wall

Dennard Scaling (1974):

i. Smaller transistors should use same power per unit area
ii. Implies more compute at same power density
iii. Ended around ∼2005: smaller transistors needed more power per unit area

Power Wall Problem:

i. Cannot fit more transistors into same area without increasing power density and needing more cooling
ii. Cannot keep increasing frequency: more frequency ⇒ more heat
iii. Processors are power-limited by thermal constraints
iv. Need to make better use of available power

Per-Processor Efficiency

Performance-per-Watt:

$$\text{Performance-per-Watt} = \frac{\text{Benchmark Score}}{\text{Processor Power}}$$

i. Depends on workload, benchmark, settings, and absolute score
ii. Performance-per-watt is not constant across frequency/power levels
iii. Often does not increase linearly with power

Processor Power Model:

$$P_{total} = P_{dynamic} + P_{static}$$
$$P_{dynamic} = kV^2 f$$

Implications:

i. $P_{dynamic}$: power used to perform operations
ii. P_{static}: power used independent of useful work
iii. Voltage V has quadratic impact on dynamic power
iv. Frequency f has linear impact and impacts performance score
v. Voltage and frequency are not independent: higher frequencies require higher voltages

Dynamic Voltage and Frequency Scaling (DVFS):

i. Modern processors adjust voltage and clock frequency dynamically
ii. Idle/simple tasks need less performance
⇒ reduce frequency and voltage
⇒ reduces power usage and heat
iii. Each frequency has a minimum safe voltage for correct operation

Heterogeneous Cores:

i. Use different core types with different power-performance curves
ii. Scheduler chooses core type based on workload demand

Datacenter Efficiency

Efficiency Metrics:

i. GFLOPs-per-Watt: compute efficiency for large benchmarks
   - Green500 ranks supercomputers
   - Measures compute power efficiency, not full facility overhead
ii. Power Usage Effectiveness:

$$\text{PUE} = \frac{\text{Total Facility Energy}}{\text{IT Equipment Energy}} = 1 + \frac{\text{Non-IT Facility Energy}}{\text{IT Equipment Energy}}$$

   - IT energy: servers, networking, storage
   - Non-IT energy: cooling, lighting, pumps, power conversion
   - Lower PUE means less overhead energy

Issues with PUE:

i. Hot climates are disadvantaged
ii. Larger compute loads can reduce PUE while increasing absolute energy use
iii. Operators may be incentivized to measure fewer overheads
iv. PUE does not measure computational efficiency directly

Reducing Datacenter Energy:

i. Improve compute efficiency:
   - Integrated accelerators
   - High-bandwidth CPU–GPU links (e.g. Grace–Hopper)
   - Unified memory / less data movement
ii. Improve facility efficiency:
   - Aisle containment: separate hot/cold air paths
   - Warm-water cooling: directly cool CPUs/GPUs

Appendix

name	prefix	multiplier
exa	E	10^{18}
peta	P	10^{15}
tera	T	10^{12}
giga	G	10^{9}
mega	M	10^{6}
kilo	K	10^{3}
milli	m	10^{-3}
micro	μ	10^{-6}
nano	n	10^{-9}
pico	p	10^{-12}

name	prefix	multiplier
exbi	Ei	2^{60}
pebi	Pi	2^{50}
tebi	Ti	2^{40}
gibi	Gi	2^{30}
mebi	Mi	2^{20}
kibi	Ki	2^{10}

Parbegin–Parend Syntax

Parbegin–parend explicitly specifies parallel regions:

i. `parbegin`: start of parallel/concurrent region
ii. `parallel`: separates statements that run concurrently
iii. `parend`: end of parallel region; spawned threads join
iv. Statements without `parallel` separation execute sequentially

```
parbegin
  do
    parbegin
      A
      parallel
      B
    parend
    C
  parallel
  D
parend
E
```

i. Inner: `A` and `B` run concurrently
ii. `C` waits for inner `A` and `B` to complete
iii. Outer: `D` runs concurrently with the inner block
iv. `E` waits for the entire outer region to complete

Special Constants

OpenMP Reduction Operations:

i. Arithmetic: `+`, `-`, `*`
ii. Logical: `&&`, `||`
iii. Bitwise: `&`, `|`, `^`
iv. Min/Max: `min`, `max`

MPI Types:

i. Integer: `MPI_INT`, `MPI_LONG`, `MPI_LONG_LONG`
ii. Floating-point: `MPI_FLOAT`, `MPI_DOUBLE`, `MPI_LONG_DOUBLE`
iii. Character: `MPI_CHAR`, `MPI_BYTE`
iv. Unsigned: `MPI_UNSIGNED`, `MPI_UNSIGNED_LONG`
v. Logical: `MPI_C_BOOL`, `MPI_CXX_BOOL`
vi. Custom: built using `MPI_Type_create_*`, then committed with `MPI_Type_commit`

MPI Reduction Operations:

i. `MPI_MAX`. `MPI_MIN`
ii. `MPI_SUM`, `MPI_PROD`
iii. Logical: `MPI_LAND`, `MPI_LOR`, `MPI_LXOR`
iv. Bitwise: `MPI_BAND`, `MPI_BOR`, `MPI_BXOR`
v. Locate: `MPI_MAXLOC`, `MPI_MINLOC`

MPI Wildcards / Constants:

i. `MPI_COMM_WORLD`: default communicator
ii. `MPI_ANY_SOURCE`: receive from any source
iii. `MPI_ANY_TAG`: receive with any tag
iv. `MPI_STATUS_IGNORE`, `MPI_STATUSES_IGNORE`: ignore status output

Shear Sort

Shear sort sorts N values on a $\sqrt{N} \times \sqrt{N}$ 2D mesh of processing elements. Shear sort is suited to a 2D mesh because communication is mostly row-wise/column-wise between neighbouring processing elements.

Layout:

i. One value per processing element
ii. Final order is snake-like:
   - Odd rows ascending
   - Even rows descending
   - Columns ascending

Algorithm:

i. Row sort: odd rows ascending, even rows descending
ii. Column sort: all columns ascending
iii. Repeat row/column phases until sorted
iv. Finish with a row sort

Complexity:

i. Number of phases: $\log_2 N + 1$
ii. Each row/column sort can use odd-even transposition sort
iii. Each phase takes $\sqrt{N}$ parallel steps
iv. Total parallel steps:

$$\sqrt{N}(\log_2 N + 1)$$